

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione (DIETI)

Corso di Ingegneria del Software – A.A. 2025/2026 – Canale 2

BugBoard26

Documentazione di Progetto

Specifica dei Requisiti, Design del Sistema e del Software, Testing

Autore: Marco Lerro – Matricola N86005403
Gruppo di lavoro: Gruppo singolo (1 componente)
Stack tecnologico: ASP.NET Core, React, PostgreSQL, Docker

20 luglio 2026

Indice

1	Introduzione	3
1.1	Scopo del documento	3
1.2	Panoramica del sistema BugBoard26	3
1.3	Funzionalità assegnate	3
1.4	Stack tecnologico	4
2	Specifica dei Requisiti Software	5
2.1	Glossario	5
2.2	Definizione degli utenti target	5
2.3	Modellazione dei casi d'uso	6
2.4	Requisiti non funzionali e di sistema	7
2.4.1	Requisiti non funzionali	7
2.4.2	Requisiti di sistema	7
2.5	Formalizzazione dei casi d'uso significativi	8
2.5.1	Caso d'uso: Creazione di una Issue	8
2.5.2	Caso d'uso: Ricerca di Issue tramite Parola Chiave	8
3	Design del Sistema	10
3.1	Architettura proposta	10
3.1.1	Criteri di design adottati	10
3.2	Scelte tecnologiche e motivazioni	10
3.2.1	Back-end: ASP.NET Core	10
3.2.2	Front-end: React	11
3.2.3	Persistenza: PostgreSQL	11
3.2.4	Containerizzazione: Docker	11
4	Design del Software	12
4.1	Schema di persistenza dei dati	12
4.2	Diagramma delle classi di design	12
4.3	Motivazione delle scelte di software design	13
4.4	Versioning	13
4.5	Report di qualità del codice	13
4.6	Codice sorgente e artefatti di build	13
5	Attività di Testing	15
5.1	Test plan	15
5.2	Test di unità automatici	15
5.3	Strategie di progettazione dei test	17
5.4	Valutazione dell'usabilità	18
5.4.1	Ispezione basata su checklist	18

5.4.2	Risultati del Questionario SUS	19
-------	--	----

1 Introduzione

1.1 Scopo del documento

Il documento costituisce la documentazione di progetto richiesta dal committente SoftEngUniNA, nell'ambito del corso di Ingegneria del Software A.A. 2025/2026. Il documento raccoglie, in un unico elaborato strutturato in capitoli, la specifica dei requisiti software, il design del sistema e del software, e la documentazione delle attività di testing relative alle funzionalità assegnate del sistema **BugBoard26**.

1.2 Panoramica del sistema BugBoard26

BugBoard26 è una piattaforma per la gestione collaborativa di issue in progetti software. Il sistema consente a team di sviluppo di segnalare problemi relativi a un progetto, monitorarne lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione.

1.3 Funzionalità assegnate

Il committente ha assegnato il seguente sottoinsieme non negoziabile di funzionalità, tra quelle elencate nella Sezione 3.1 del documento dell'incarico:

ID	Descrizione sintetica
1	Sistema di autenticazione basato su email/password. Account amministratore precaricato con credenziali di default; gli amministratori possono creare ulteriori utenze (normali o di amministrazione).
2	Tutti gli utenti autenticati possono segnalare una issue (titolo, descrizione, priorità opzionale, allegato immagine opzionale). Le issue possono essere di tipo <i>question</i> , <i>bug</i> , <i>documentation</i> , <i>feature</i> e nascono nello stato <i>todo</i> .
3	Vista riepilogativa delle issue, con filtro e ordinamento per tipologia, stato, priorità e altri parametri rilevanti.
5	Sezione commenti associata a ciascuna issue, in cui tutti gli utenti possono lasciare messaggi, aggiornamenti o richieste di chiarimento.
9	Gli utenti possono modificare solo le issue a loro assegnate; gli amministratori hanno accesso completo in scrittura.
11	Funzione di ricerca che consente di trovare issue in base a parole chiave.
18	Gli amministratori possono impostare scadenze opzionali per la risoluzione di una issue.

1.4 Stack tecnologico

Il sistema è realizzato come applicazione web distribuita, rispettando il vincolo di separazione netta tra back-end e front-end richiesto dalla commessa:

- **Back-end:** ASP.NET Core Web API (C#), che espone i servizi applicativi tramite API REST, completamente indipendente dal front-end.
- **Front-end:** applicazione web Single Page Application realizzata in React, che comunica con il back-end esclusivamente tramite le API REST esposte.
- **Persistenza:** PostgreSQL, gestito tramite Entity Framework Core come ORM.
- **Containerizzazione:** Docker e Docker Compose per la gestione dei container di back-end e database, a supporto della portabilità e del deployment.

Le motivazioni dettagliate delle scelte tecnologiche sono riportate nel Capitolo 3.

2 Specifica dei Requisiti Software

2.1 Glossario

Termine	Definizione
Issue	Elemento informativo che rappresenta una segnalazione all'interno del sistema (domanda, bug, problema di documentazione o richiesta di funzionalità).
Tipo di Issue	Categoria di una issue: <i>question, bug, documentation, feature</i> .
Stato	Fase del ciclo di vita di una issue (es. <i>todo, in progress, done</i>).
Priorità	Attributo opzionale di una issue che ne indica l'urgenza (es. bassa, media, alta).
Commento	Messaggio testuale associato a una issue, inserito da un utente autenticato.
Scadenza (deadline)	Data opzionale, impostabile solo da un amministratore, entro cui una issue dovrebbe essere risolta.
Utente Standard	Utente autenticato con permessi limitati: può creare issue, commentare, modificare solo le issue a lui assegnate.
Amministratore	Utente con permessi estesi: può creare utenze, ha accesso completo in scrittura a tutte le issue, può impostare scadenze.
Autenticazione	Processo di verifica dell'identità di un utente tramite coppia email/password.
Token di sessione (JWT)	Token firmato digitalmente utilizzato per mantenere lo stato di autenticazione tra front-end e back-end senza sessione stateful lato server.
API REST	Interfaccia di programmazione esposta dal back-end, basata sui principi architetturali REST, tramite cui il front-end accede ai dati e alle funzionalità.

2.2 Definizione degli utenti target

Il sistema individua tre categorie di attori:

- **Amministratore:** utente con responsabilità di gestione del sistema e del team. Crea le utenze, ha accesso in scrittura completo a tutte le issue e può impostarne le scadenze.
- **Utente Standard:** membro del team di sviluppo o segnalatore, che utilizza il sistema per segnalare problemi, consultarli, commentarli e aggiornare lo stato delle issue a lui assegnate.
- **Stakeholder:** utente che è limitato alla sola visualizzazione delle issue.

Tutte le categorie devono autenticarsi prima di poter accedere alle funzionalità del sistema; non sono previsti accessi anonimi.

2.3 Modellazione dei casi d'uso

La Figura 2.1 riporta il diagramma dei casi d'uso relativo alle funzionalità assegnate. Il diagramma è stato realizzato in Visual Paradigm; il file, come richiesto, è disponibile nel repository di progetto.

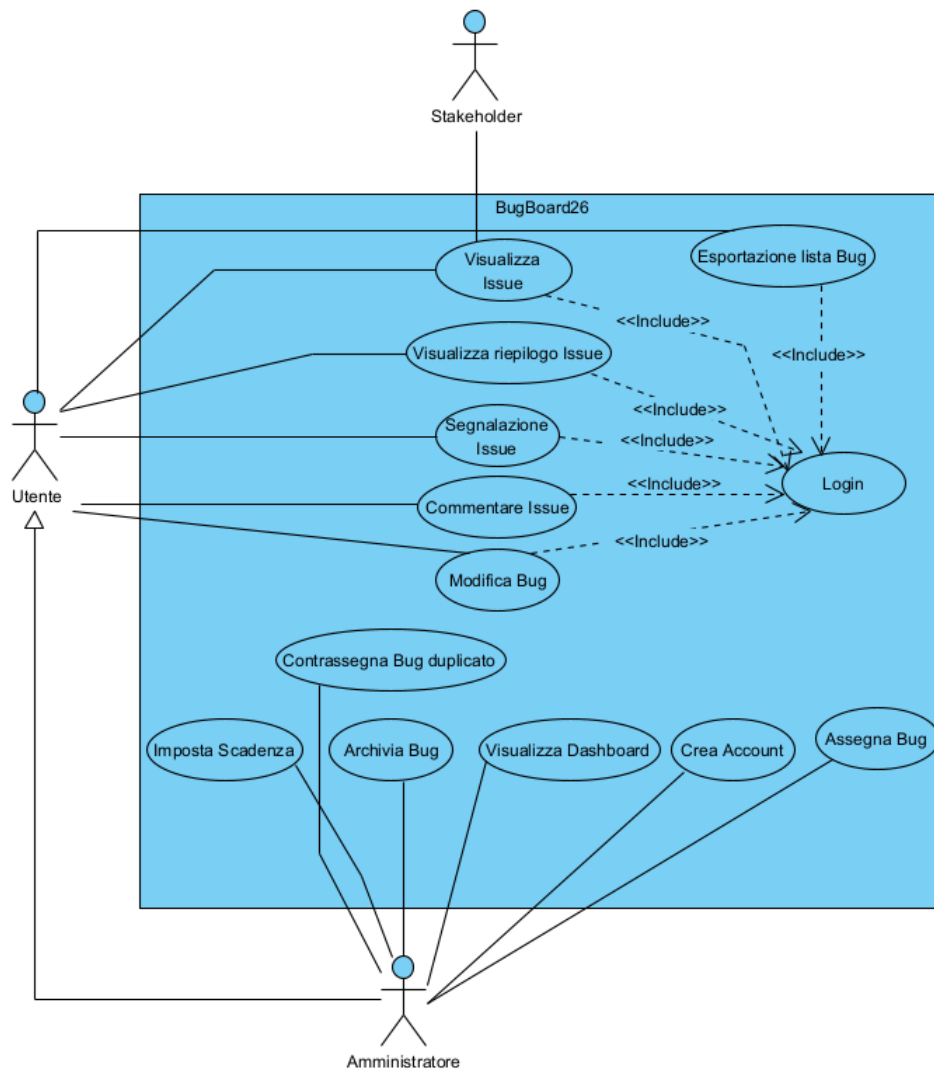


Figura 2.1: Diagramma dei casi d'uso (bozza) – funzionalità assegnate

2.4 Requisiti non funzionali e di sistema

2.4.1 Requisiti non funzionali

Categoria	Descrizione
Sicurezza	Le password sono memorizzate esclusivamente in forma hash; l'autenticazione avviene tramite token JWT firmato; tutte le comunicazioni client-server dovrebbero avvenire su HTTPS in ambiente di produzione.
Autorizzazione	Ogni endpoint del back-end applica controlli di autorizzazione basati sul ruolo (Amministratore/Utente Standard) e sulla proprietà della risorsa (es. una issue è modificabile da un utente standard solo se a lui assegnata).
Usabilità	L'interfaccia deve permettere di creare una issue e trovarne una tramite ricerca in un numero ridotto di passaggi (idealmente ≤ 3 click dalla schermata principale).
Affidabilità	Il sistema deve gestire correttamente input non validi restituendo messaggi di errore comprensibili, senza causare stati inconsistenti nel database.
Portabilità	L'intero back-end e il database devono poter essere eseguiti tramite container Docker, riproducibili su qualunque host che supporti Docker Engine.
Manutenibilità	L'architettura a livelli e l'uso di pattern di astrazione (Repository, DTO, Dependency Injection) devono consentire l'estensione futura con nuove funzionalità (es. quelle non assegnate nella traccia) senza modifiche invasive al codice esistente.
Performance	Le operazioni di lista/ricerca/filtro sulle issue devono restituire risposta in tempi ragionevoli anche al crescere del numero di record, tramite paginazione e query indicizzate lato database.

2.4.2 Requisiti di sistema

- Back-end: .NET 8 (o successivo), eseguito in container Docker Linux.
- Database: PostgreSQL 15+ (o successivo), eseguito in container Docker separato.
- Front-end: applicazione React (Node.js 18+ in fase di build), compatibile con i principali browser desktop aggiornati (Chrome, Firefox, Edge).
- Comunicazione: API REST su protocollo HTTP/HTTPS, formato dei payload JSON.

2.5 Formalizzazione dei casi d'uso significativi

Come richiesto, seguono due casi d'uso tramite template di A. Cockburn, seguiti da Mock-Up visuale delle interfacce coinvolte.

2.5.1 Caso d'uso: Creazione di una Issue

UC-01: Creazione di una Issue	
Attore primario	Utente autenticato (Utente Standard o Amministratore)
Scopo	Consentire a un utente autenticato di segnalare un nuovo problema o richiesta al sistema
Livello	Obiettivo utente (user-goal)
Stakeholder e interessi	Utente: vuole registrare rapidamente una segnalazione. Amministratore/Team: vuole ricevere segnalazioni complete e correttamente categorizzate.
Precondizioni	L'utente ha effettuato l'accesso al sistema con credenziali valide.
Postcondizioni	Una nuova issue è persistita nel sistema con stato iniziale <i>todo</i> , associata all'utente segnalante e con timestamp di creazione registrato nella cronologia.
Trigger	L'utente seleziona l'azione "Nuova Issue" dall'interfaccia.
Scenario principale	1. L'utente seleziona "Nuova Issue". 2. Il sistema mostra il form di creazione (titolo, descrizione, tipo, priorità opzionale, allegato opzionale). 3. L'utente compila titolo, descrizione e seleziona il tipo (question/bug/documentation/feature). 4. L'utente conferma l'invio. 5. Il sistema valida i dati obbligatori (titolo e descrizione non vuoti). 6. Il sistema crea la issue con stato <i>todo</i>, la associa all'utente autenticato e la salva. 7. Il sistema mostra conferma e reindirizza alla vista di dettaglio della issue creata.
Estensioni	3a. L'utente specifica anche una priorità: il valore viene incluso nella issue. 3b. L'utente allega un'immagine: il sistema effettua l'upload e associa il riferimento al file alla issue. 5a. Titolo o descrizione mancanti: il sistema mostra un messaggio di errore e non salva la issue; si ritorna al passo 3. 5b. Formato o dimensione dell'allegato non valido: il sistema segnala l'errore e consente di riprovare senza perdere i dati già inseriti.
Requisiti speciali	Il campo tipo è obbligatorio con valore di default <i>question</i> se non selezionato esplicitamente.
Frequenza	Alta: è l'operazione più frequente svolta dagli utenti standard.

2.5.2 Caso d'uso: Ricerca di Issue tramite Parola Chiave

Nuova issue

Titolo

Descrizione

Tipo

[scegli tipo ▼]

Priorità (opz.)

[--- ▼]

Allegato (opz.)

[Scegli file...]

Annulla

Crea Issue

Figura 2.2: Mock-up (bozza) – Form di creazione Issue

UC-02: Ricerca di Issue tramite Parola Chiave	
Attore primario	Utente autenticato (Utente Standard o Amministratore)
Scopo	Consentire all’utente di individuare rapidamente una o più issue in base a una parola chiave
Livello	Obiettivo utente (user-goal)
Stakeholder e interessi	Utente: vuole ritrovare rapidamente issue rilevanti senza scorrere l’intero elenco.
Precondizioni	L’utente ha effettuato l’accesso al sistema. Esiste almeno una issue nel sistema.
Garanzia di successo (postcondizioni)	Viene mostrato all’utente l’elenco delle issue il cui titolo e/o descrizione contengono la parola chiave inserita.
Trigger	L’utente digita un termine nella barra di ricerca.
Scenario principale	1. L’utente accede alla vista riepilogativa delle issue. 2. L’utente digita una parola chiave nel campo di ricerca. 3. Il sistema invia la richiesta di ricerca al back-end. 4. Il back-end interroga il database filtrando le issue per corrispondenza (parziale, case-insensitive) su titolo e descrizione. 5. Il sistema restituisce e mostra l’elenco delle issue corrispondenti, mantenendo eventuali filtri/ordinamenti già attivi.
Estensioni	5a. Nessuna issue corrisponde alla parola chiave: il sistema mostra un messaggio di “nessun risultato trovato”. 2a. L’utente cancella il testo di ricerca: il sistema ripristina l’elenco completo (con eventuali altri filtri attivi).
Requisiti speciali	La ricerca deve restituire risultati in tempo utile (percepito come istantaneo) anche con centinaia di issue in database, grazie a indicizzazione lato database.
Frequenza	Media-alta.

Elenco Issue

Cerca per parola chiave...

9

Filtri

Ordina

Login fallisce su Safari

[bug]

todo

Login fallisce su Safari

[bug]

todo

3 Design del Sistema

3.1 Architettura proposta

Dato il vincolo imposto dalla commessa, il sistema è organizzato secondo un'architettura distribuita a due macro-componenti indipendenti: un back-end che espone API REST e un front-end SPA che si occupa esclusivamente dei servizi via rete. Il back-end è inoltre organizzato internamente secondo un'architettura a livelli, per favorire la separazione delle responsabilità e la manutenibilità.

3.1.1 Criteri di design adottati

- **Separazione back-end/front-end:** il front-end non ha alcuna conoscenza dell'implementazione interna del back-end e comunica esclusivamente tramite API REST versionate; questo consente di sostituire l'uno indipendentemente dall'altro.
- **Architettura a livelli nel back-end:** i controller REST (Presentation) delegano la logica di business ai Services (Application), che operano su Entità di Dominio tramite Repository (Persistenza).
- **Dependency Injection:** ASP.NET Core fornisce nativamente un container di Dependency Injection, utilizzato per iniettare Repository e Services, favorendo testabilità e sostituibilità dei componenti (es. con mock nei test di unità).
- **Data Transfer Object (DTO):** le entità di dominio non vengono mai esposte direttamente tramite le API; si utilizzano DTO dedicati per le richieste e le risposte, disaccoppiando il modello di persistenza dal contratto esposto al client.
- **Stato centralizzato lato server:** tutti i dati persistenti (issue, commenti, utenti, scadenze) sono gestiti unicamente dal back-end e dal database PostgreSQL; il front-end mantiene solo stato applicativo temporaneo (form, filtri correnti, token di sessione).

3.2 Scelte tecnologiche e motivazioni

3.2.1 Back-end: ASP.NET Core

ASP.NET Core è stato scelto per la realizzazione del back-end in quanto:

- è un framework Object-Oriented (C#) maturo, con tipizzazione statica che riduce gli errori a runtime;
- permette di creare facilmente API REST usando strumenti già integrati, come i Controller e i middleware;
- si integra nativamente con Entity Framework Core, un ORM completo per l'accesso a PostgreSQL, e con meccanismi di autenticazione basati su JWT;

- è multiplatforma e si presta naturalmente alla containerizzazione tramite immagini Docker ufficiali Microsoft.

3.2.2 Front-end: React

React è stato scelto per il front-end in quanto:

- consente di realizzare un'interfaccia a componenti riutilizzabili, adatta a una Single Page Application che utilizza esclusivamente API REST;
- il Virtual DOM garantisce buone performance di rendering anche con aggiornamenti frequenti dell'interfaccia (es. lista issue filtrata in tempo reale);
- l'ecosistema (React Router per la navigazione, Axios per le chiamate HTTP, eventuali librerie di state management) è ampio e maturo;
- l'architettura a componenti favorisce la completa indipendenza dal back-end, comunicando con esso solo tramite chiamate HTTP verso le API esposte.

3.2.3 Persistenza: PostgreSQL

PostgreSQL è stato scelto come sistema di gestione di basi di dati relazionale in quanto:

- garantisce proprietà ACID, fondamentali per la coerenza dei dati relativi a issue, assegnazioni e commenti;
- offre un ottimo supporto nell'ecosistema .NET tramite il provider Npgsql per Entity Framework Core;
- è open source, gratuito e ampiamente supportato dai principali provider di public cloud (Azure Database for PostgreSQL, AWS RDS);
- supporta indicizzazione full-text, utile per l'implementazione efficiente della funzionalità di ricerca per parola chiave (requisito 11).

3.2.4 Containerizzazione: Docker

Il back-end e il database sono containerizzati tramite Docker e orchestrati con Docker Compose. Questo garantisce:

- riproducibilità dell'ambiente di esecuzione indipendentemente dalla macchina host;
- possibilità di deployment su servizi di public cloud (es. Azure Container Apps, AWS ECS) in fase di discussione del progetto;
- isolamento tra il container applicativo e il container del database, ciascuno con il proprio ciclo di vita.

Nota: nessuna logica applicativa o di persistenza è delegata a servizi MBaaS esterni (es. Firebase, Supabase): tutta la logica di business risiede nel back-end ASP.NET Core e tutti i dati sono persistiti su un'istanza PostgreSQL controllata dal team di progetto, come espresso dal vincolo della commessa.

4 Design del Software

4.1 Schema di persistenza dei dati

Lo schema seguente mostra la persistenza dei dati nel database, implementato con PostgreSQL:

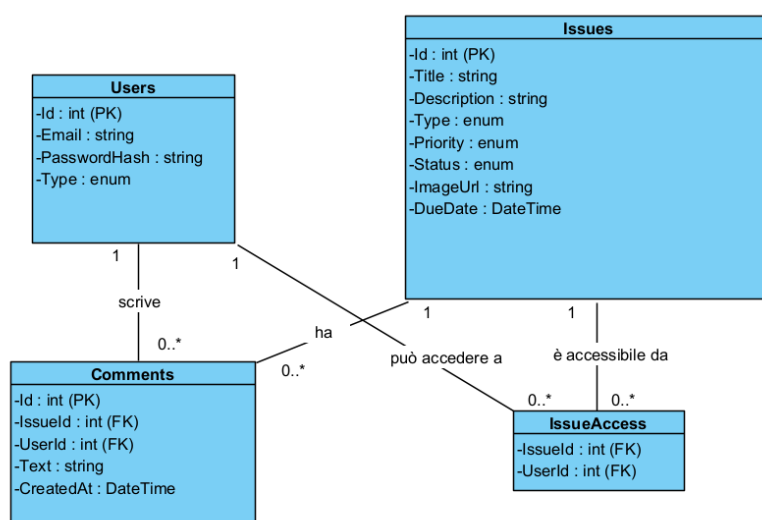
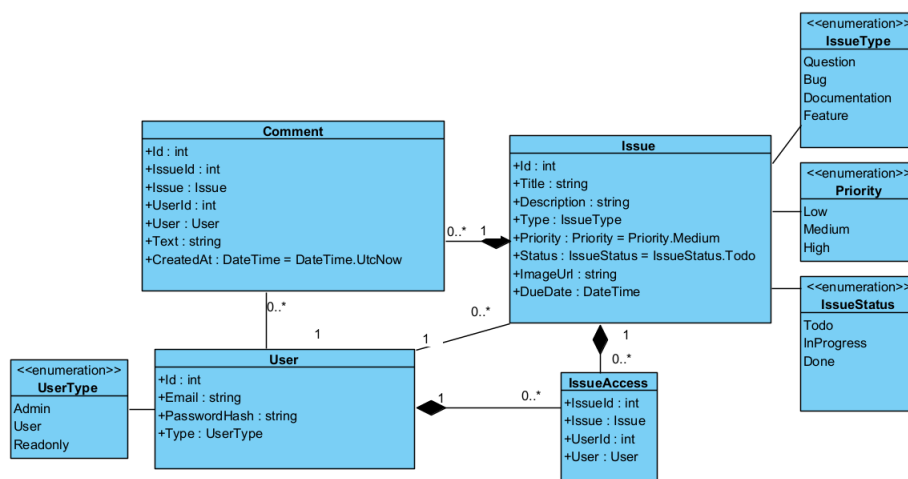


Figura 4.1: Schema concettuale della persistenza dati (bozza)

4.2 Diagramma delle classi di design

Il seguente diagramma mostra le classi implementate coinvolte nel back-end:



4.3 Motivazione delle scelte di software design

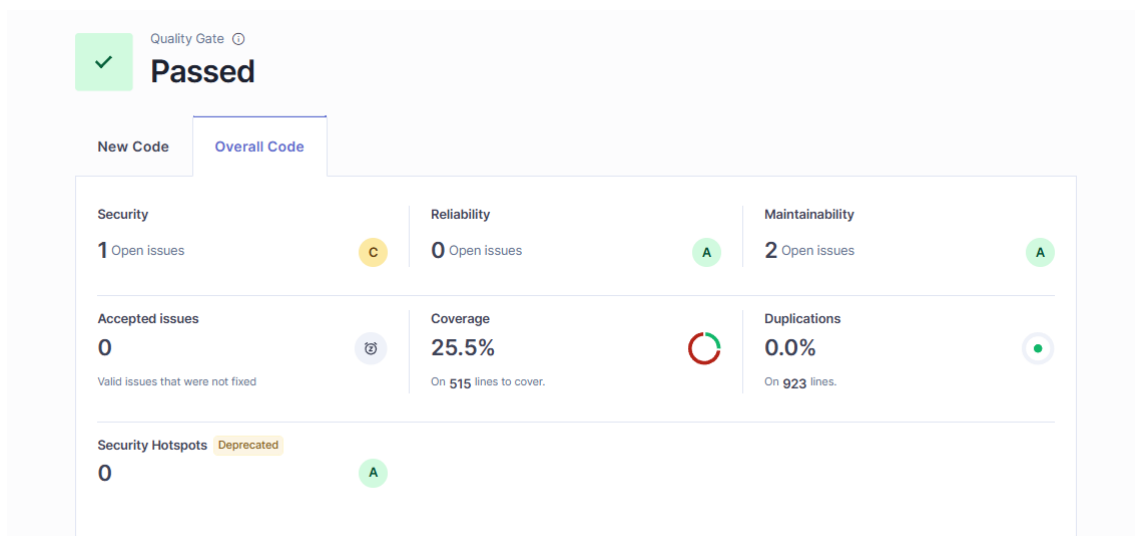
- **Separazione delle responsabilità:** le principali entità sono state distinte per mantenere la separazione netta tra il dominio dell'utente, della issue e della discussione (User, Issue, Comment)
- **Gestione della relazione tra utente e issues** la classe associativa **IssueAccess** è stata realizzata per gestire la relazione multi-a-molti tra User e Issue: anziché inserire riferimenti rigidi nelle entità, la logica dei permessi è stata isolata in un punto unico.
- **Tipizzazione forte:** è stato fatto largo uso di **Enumeration** (IssueType, Priority, Status) per garantire la type safety: il sistema non accetta stati o priorità non validi, riducendo così il rischio di errori al runtime.

4.4 Versioning

Il codice sorgente del progetto è gestito tramite Git e ospitato su GitHub. <https://github.com/lerrooooo/bugboard26>

4.5 Report di qualità del codice

Il back-end è analizzato tramite SonarQube per la verifica di code smell, duplicazioni e complessità ciclomatica. Di seguito è presente il report:



Il problema di security è dato dalle credenziali hardcoded che in fase di shipping potrebbero essere implementate diversamente. La coverage indica la copertura dei tre test effettuati.

4.6 Codice sorgente e artefatti di build

Il codice sorgente completo, comprensivo di **Dockerfile** per il back-end, **docker-compose.yml** per l'orchestrazione back-end/database, e file di build automatica, è disponibile nel repository GitHub indicato in Sezione 4.4.

Listing 4.1: Estratto indicativo di **docker-compose.yml**

```
1 services:
2   backend:
3     build: ./backend
```

```
4     ports:
5       - "5000:8080"
6     depends_on:
7       - db
8     environment:
9       - ConnectionStrings__Default=Host=db;Database=bugboard26;...
10 db:
11   image: postgres:16
12   environment:
13     - POSTGRES_DB=bugboard26
14     - POSTGRES_PASSWORD=[inserire]
15   volumes:
16     - dbdata:/var/lib/postgresql/data
17 volumes:
18   dbdata:
```

5 Attività di Testing

5.1 Test plan

Il test plan riguarda, come richiesto dalla commessa, tre metodi con almeno due parametri. I metodi testati sono: `CreateIssue` (7 parametri), `UpdateIssueDto` (4 parametri), `AddComment` (2 parametri).

5.2 Test di unità automatici

Di seguito è riportato il codice dei test:

Listing 5.1: Estratto di test xUnit per `UpdateIssue`

```
1  using System.Security.Claims;
2  using Microsoft.AspNetCore.Http;
3  using Microsoft.AspNetCore.Mvc;
4  using Microsoft.EntityFrameworkCore;
5  using Xunit;
6  using BugBoard26_BackEnd.Controllers;
7  using BugBoard26_BackEnd.Data;
8  using BugBoard26_BackEnd.Models;
9  using BugBoard26_BackEnd.Models.Dtos;
10 using BugBoard26_BackEnd.Models.Enums;
11
12 namespace BugBoard26_BackEnd.Tests
13 {
14     public class Tests
15     {
16         // crea un db finto in memoria per i test
17         private AppDbContext GetDb()
18         {
19             var opt = new DbContextOptionsBuilder<AppDbContext>()
20                 .UseInMemoryDatabase(Guid.NewGuid().ToString())
21                 .Options;
22             return new AppDbContext(opt);
23         }
24
25         // simula un utente loggato con un certo ruolo
26         private void Login(ControllerBase c, int id, string role)
27         {
28             var claims = new List<Claim>
29             {
30                 new(ClaimTypes.NameIdentifier, id.ToString()),
31                 new(ClaimTypes.Role, role)
32             };
33             var user = new ClaimsPrincipal(new ClaimsIdentity(claims, "test"));
34             c.ControllerContext = new ControllerContext
35             {
```



```
36         HttpContext = new DefaultHttpContext{User = user}
37     };
38 }
39
40 //test 1: creazione issue da admin
41 [Fact]
42 public async Task CreateIssue_Admin_Funziona()
43 {
44     var db = GetDb();
45     var ctrl = new IssuesController(db);
46     Login(ctrl, 1, "Admin");
47
48     var res = await ctrl.CreateIssue(
49         "Bug login",
50         "il login non va",
51         IssueType.Bug,
52         Priority.High,
53         new List<int>{2, 3},
54         "2026-08-01",
55         null);
56
57     var ok = Assert.IsType<OkObjectResult>(res);
58     var issue = Assert.IsType<Issue>(ok.Value);
59
60     Assert.Equal("Bug login", issue.Title);
61     Assert.Equal(2, issue.AccessEntries.Count);
62 }
63
64 // test 2: update issue da utente senza permessi -> deve dare forbid
65 [Fact]
66 public async Task UpdateIssue_UtenteSenzaAccesso_DaForbid()
67 {
68     var db = GetDb();
69     var issue = new Issue { Title = "test", Type = IssueType.Bug };
70     db.Issues.Add(issue);
71     await db.SaveChangesAsync();
72
73     var ctrl = new IssuesController(db);
74     Login(ctrl, 99, "User"); // utente 99 non ha accesso
75
76     var dto = new IssuesController.UpdateIssueDto
77     {
78         Title = "modificato",
79         Type = IssueType.Feature,
80         Priority = Priority.Low,
81         Status = IssueStatus.InProgress
82     };
83
84     var res = await ctrl.UpdateIssue(issue.Id, dto);
85
86     Assert.IsType<ForbidResult>(res);
87 }
88
89 //test 3: aggiunta commento
90 [Fact]
91 public async Task AddComment_Funziona()
92 {
93     var db = GetDb();
94     var user = new User { Email = "marco@test.com", PasswordHash = "x"
95         " };
96     var issue = new Issue { Title = "test", Type = IssueType.Bug };
97     db.Users.Add(user);
98     db.Issues.Add(issue);
```

```
98         await db.SaveChangesAsync();
99
100        var ctrl = new CommentsController(db);
101        Login(ctrl, user.Id, "User");
102
103        var dto = new CreateCommentDto { Text = "  prova commento  " };
104
105        var res = await ctrl.AddComment(issue.Id, dto);
106
107        Assert.IsType<OkObjectResult>(res);
108        var salvato = await db.Comments.FirstAsync();
109        Assert.Equal("prova commento", salvato.Text);
110    }
111 }
112 }
```

5.3 Strategie di progettazione dei test

- **Metodologia Generale e Isolamento:** La suite di test per il backend di BugBoard26 è stata progettata seguendo i principi del Testing di Unità Isolata. Per garantire l'indipendenza dei test e la ripetibilità dell'esecuzione, è stato adottato un Database in-memory, generando un codice di identificazione univoco (`Guid.NewGuid()`). Al posto di effettuare l'autenticazione attraverso token JWT, è stata simulata generando un `ClaimsPrincipal`.
- **Criteri di Progettazione e Copertura:** per la selezione dei casi di test, è stato adottato un approccio ibrido che combina tecniche Black-Box (basate sulle specifiche) e White-Box (basate sulla struttura del codice) e sono stati scelti inoltre 3 metodi, come descritto nella sezione 5.1:
 - Metodo `CreateIssue`:
 - * **Tecnica Black-Box:**
Variabile considerata: Ruolo Utente.
Partizioni individuate: Admin, User, Guest/Non autenticato.
Classe coperta dal test: Admin con dati di input validi.
 - * **Tecnica White-Box:**
Il test attraversa il flusso principale di esecuzione e garantisce che le istruzioni vengano mappate e salvate sul database.
 - Metodo `UpdateIssue`:
 - * **Tecnica Black-Box:**
Variabile considerata: Permessi di accesso alla risorsa `Issue`.
Partizioni individuate: Utente Autore/Assegnatario, Utente estraneo.
Classe coperta dal test: Utente estraneo (ID 99), operazione non effettuabile dall'utente.
 - * **Tecnica White-Box:**
Il test è mirato per coprire la specifica condizione `if(!hasAccess)` che verifica i permessi.
 - Metodo `AddComment`:
 - * **Tecnica Black-Box:**
Variabile considerata: Formattazione della stringa di input.

Partizioni individuate: Stringhe formattate correttamente, Stringhe con spazi bianchi in eccesso agli estremi.

Classe coperta dal test: Stringhe con spazi bianchi (" prova commento ").

* **Tecnica White-Box:**

Il test non si limita a verificare la risposta HTTP, ma interroga direttamente il database in-memory post-esecuzione per assicurarsi che il dato abbia subito la corretta trasformazione (`Trim()`) prima della memorizzazione sul disco.

5.4 Valutazione dell'usabilità

A complemento delle attività di testing funzionale, è stata condotta una valutazione preliminare dell'usabilità dell'interfaccia, secondo due modalità:

- **Ispezione basata su checklist:** revisione sistematica dell'interfaccia rispetto alle euristiche di Nielsen (visibilità dello stato del sistema, corrispondenza tra sistema e mondo reale, prevenzione degli errori, ecc.), applicata alle schermate di creazione issue e ricerca.
- **Test con utenti:** sessioni di osservazione con utenti reali/potenziati a cui viene chiesto di completare compiti rappresentativi (creare una issue, cercarne una per parola chiave, aggiungere un commento), seguite da un breve questionario di gradimento (es. System Usability Scale).

Le domande poste al questionario sono state le seguenti (3 utenti analizzati):

1. Penso che mi piacerebbe usare BugBoard26 frequentemente.
2. Ho trovato BugBoard26 inutilmente complesso.
3. Ho trovato BugBoard26 facile da usare.
4. Penso che avrei bisogno del supporto di un tecnico per poter usare BugBoard26.
5. Ho trovato che le varie funzioni di BugBoard26 fossero ben integrate.
6. Ho trovato troppa incoerenza in BugBoard26.
7. Immagino che la maggior parte delle persone imparerebbe a usare BugBoard26 molto rapidamente.
8. Ho trovato BugBoard26 molto macchinoso (o ingombrante) da usare.
9. Mi sono sentito molto sicuro nell'usare BugBoard26.
10. Ho dovuto imparare molte cose prima di poter iniziare a usare BugBoard26.

5.4.1 Ispezione basata su checklist

Di seguito è riportata la tabella con la checklist verificata dal gruppo di lavoro.

Euristica di Nielsen	Osservazione (BugBoard26)	Severità
1. Visibilità dello stato del sistema	Durante il caricamento delle Issue e la fase di login, è presente un feedback visivo immediato (scritta accesso in corso). I messaggi di successo compaiono in tempo reale.	0
2. Corrispondenza con il mondo reale	La terminologia usata (Issue, Bug, Priority, Assignee) è standard e familiare per il target di riferimento (sviluppatori e project manager).	0
3. Controllo e libertà dell'utente	Nei form di creazione e modifica delle Issue è sempre presente e ben visibile un pulsante di annullamento (X) per interrompere l'azione senza salvare.	0
4. Coerenza e standard	I colori e i layout sono coerenti in tutto il sistema: le azioni primarie sono blu, le azioni distruttive (es. eliminazione) sono rosse e le modifiche gialle.	0
5. Prevenzione degli errori	La pagina di login restituisce in tempo reale il mancato login dell'utente. Il pulsante "Salva Commento" non rimane disabilitato nonostante l'utente non inserisca testo valido.	1
6. Riconoscimento anziché ricordo	Per assegnare una Issue, l'utente seleziona i membri da un menu a tendina visivo, senza dover ricordare a memoria gli ID o le email dei colleghi. Tuttavia se la lista è lunga potrebbe risultare scomoda la navigazione	1
7. Flessibilità ed efficienza	<i>Criticità:</i> Il sistema non supporta attualmente scorciatoie da tastiera (es. Ctrl+Enter per salvare rapidamente), costringendo gli utenti esperti all'uso prolungato del mouse.	2
8. Design estetico e minimalista	La Dashboard mostra solo le informazioni essenziali. I dettagli lunghi sono nascosti e accessibili solo su richiesta cliccando sulla singola Issue.	0
9. Aiuto con gli errori	<i>Criticità:</i> Se il server va offline, compare un messaggio generico "Errore di rete" invece di suggerire all'utente di ricaricare la pagina più tardi.	1
10. Aiuto e documentazione	<i>Criticità:</i> Manca un tooltip esplicativo vicino al campo "Tipo Issue" per spiegare ai nuovi utenti la differenza tecnica tra "Task" e "Feature".	1

Tabella 5.1: Valutazione euristica completa dell'interfaccia

5.4.2 Risultati del Questionario SUS

Al termine delle sessioni di osservazione pratica, è stato somministrato agli utenti il questionario standardizzato **System Usability Scale (SUS)** per ottenere una metrica quantitativa del grado di usabilità percepita.

Il questionario è composto da 10 affermazioni (5 in forma positiva e 5 in forma negativa) valutate

su una scala Likert da 1 (Fortemente in disaccordo) a 5 (Fortemente d'accordo).

La tabella seguente riassume le risposte grezze fornite dai 3 utenti coinvolti nel test:

Utente	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
Utente 1	4	2	3	1	4	2	5	1	5	1
Utente 2	4	3	4	2	3	2	4	2	4	2
Utente 3	5	2	4	1	4	2	5	2	4	1

Tabella 5.2: Risposte grezze al questionario SUS (scala 1-5)

Calcolo del Punteggio

Il punteggio SUS non è una semplice somma matematica, ma viene normalizzato su una scala da 0 a 100 applicando il seguente algoritmo ufficiale:

- Per le domande dispari (positive): *Valore della risposta - 1*
- Per le domande pari (negative): *5 - Valore della risposta*

La somma dei valori così convertiti viene infine moltiplicata per il coefficiente **2.5**.

Applicando la formula ai dati raccolti, si ottengono i seguenti punteggi per i singoli utenti:

- **Utente 1:** $34 \times 2.5 = 85$
- **Utente 2:** $28 \times 2.5 = 70$
- **Utente 3:** $34 \times 2.5 = 85$

Verdetto Finale

Il punteggio SUS medio ottenuto dal sistema BugBoard26 è pari a **80/100**. Considerando che la letteratura scientifica indica 68 come punteggio medio di accettabilità, il valore di 80 posiziona l'applicativo in una fascia di **Eccellenza (Grade A-)**. Questo risultato conferma che l'interfaccia risulta altamente intuitiva e che le violazioni euristiche minori riscontrate non inficiano l'efficienza d'uso complessiva del software.